

# Logos Language Model Efficiency Lab

## Experiment 1 Report: Weight Tying

*Scientific comparison against the decoder-only GPT baseline*

Run: logos\_exp01\_weight\_tying

### Executive Summary

This report documents the first alternate method tested after the Logos baseline: weight tying. The purpose of this experiment was to test whether sharing the input token embedding matrix with the output language-model head could reduce parameter count and checkpoint size while preserving language-modeling performance. Only one major variable was changed from the baseline, making this a clean scientific comparison. The experiment succeeded: parameters decreased from 32.16M to 19.24M, and checkpoint size decreased from 122.71 MB to 73.44 MB, while final validation loss remained nearly unchanged.

| Result Area          | Main Finding                                                        |
|----------------------|---------------------------------------------------------------------|
| Parameter efficiency | Improved strongly: 40.16% fewer parameters                          |
| Storage efficiency   | Improved strongly: checkpoint reduced by 40.15%                     |
| Validation quality   | Almost unchanged: 5.7634 baseline vs 5.7693 weight tying            |
| Training speed       | Improved: approximately 172k tokens/sec vs 159k tokens/sec baseline |
| Energy use           | Improved: 9.87 Wh vs 10.78 Wh baseline                              |
| Generation latency   | Worse in this run: 12.64 ms/token vs 10.20 ms/token baseline        |

### 1. Scientific Setup

The baseline had already established a decoder-only GPT-style causal language model trained from scratch on a 10,000-sample OpenWebText subset using the GPT-2 tokenizer. This experiment used the same training task, dataset, optimizer, number of iterations, sequence length, embedding size, depth, number of heads, dropout, and random seed. The independent variable was weight tying.

| Scientific Component      | Description                                                                                                     |
|---------------------------|-----------------------------------------------------------------------------------------------------------------|
| Control condition         | Original Logos decoder-only GPT baseline                                                                        |
| Experimental condition    | Same model with tied token embedding and LM head weights                                                        |
| Independent variable      | Weight tying enabled                                                                                            |
| Controlled variables      | Dataset, tokenizer, seed, model depth, heads, embedding dimension, batch size, iterations, optimizer, scheduler |
| Primary dependent metrics | Parameter count, validation loss, perplexity, checkpoint size, VRAM, speed, latency, and energy                 |
| Hypothesis                | Weight tying will reduce model size while keeping validation loss close to the baseline                         |

## 2. Method: Weight Tying

In the baseline architecture, the input token embedding matrix and the output projection matrix were separate. The input embedding maps token IDs into hidden vectors, while the output language-model head maps hidden vectors back to vocabulary logits. Because both matrices represent the same vocabulary space, they can share weights.

```
Baseline:  
input token embedding != output LM head
```

```
Weight tying:  
input token embedding == output LM head
```

The implemented model used an output head without bias and explicitly assigned the LM head weight tensor to the token embedding weight tensor. The notebook confirmed that weight tying was active using a direct memory pointer equality check.

```
self.lm_head = nn.Linear(N_EMBED, vocab_size, bias=False)  
self.lm_head.weight = self.token_embedding.weight
```

```
Weight tying active: True
```

## 3. Architecture and Training Configuration

| Configuration Item  | Value                                        |
|---------------------|----------------------------------------------|
| Model type          | Decoder-only GPT-style causal language model |
| Experiment type     | Weight tying                                 |
| Tokenizer           | GPT-2 tokenizer                              |
| Dataset             | OpenWebText subset                           |
| Dataset samples     | 10,000 text samples                          |
| Vocabulary size     | 50,257                                       |
| Total tokens        | 11,153,991                                   |
| Train tokens        | 10,038,591                                   |
| Validation tokens   | 1,115,400                                    |
| Layers              | 8                                            |
| Attention heads     | 8                                            |
| Embedding dimension | 256                                          |
| Context length      | 256 tokens                                   |
| Dropout             | 0.2                                          |
| Batch size          | 64                                           |
| Max iterations      | 1,500                                        |
| Optimizer           | AdamW                                        |
| Learning rate       | 3e-4                                         |
| Weight decay        | 0.1                                          |
| Gradient clipping   | 1.0                                          |
| Mixed precision     | Enabled                                      |

## 4. Hardware and Software Environment

| Item                 | Value                          |
|----------------------|--------------------------------|
| Device               | CUDA GPU                       |
| GPU                  | NVIDIA RTX 6000 Ada Generation |
| Available GPU memory | 47.99 GB                       |
| CUDA version         | 12.1                           |
| PyTorch version      | 2.5.1+cu121                    |

## 5. Weight Tying Run Results

| Metric                      | Value                              |
|-----------------------------|------------------------------------|
| Run name                    | logos_exp01_weight_tying           |
| Parameters                  | 19,243,776                         |
| Trainable parameters        | 19,243,776                         |
| Weight tying active         | True                               |
| Best iteration              | 1,500                              |
| Best validation loss        | 5.7668                             |
| Final training loss         | 5.6234                             |
| Final validation loss       | 5.7693                             |
| Final validation perplexity | 320.32                             |
| Checkpoint size             | 73.44 MB                           |
| Peak VRAM                   | 10,690.32 MB, about 10.69 GB       |
| Training time               | 144.17 seconds, about 2.40 minutes |
| Training speed              | 172,070 tokens/sec                 |
| Generation speed            | 79.08 tokens/sec                   |
| Generation latency          | 12.64 ms/token                     |
| Average GPU power           | 249.47 W                           |
| Estimated GPU energy        | 9.87 Wh                            |

## 6. Training Behavior

Validation loss decreased steadily throughout the run, and the best checkpoint was saved at the final iteration. This means the model was still learning at the end of the fixed 1,500-iteration budget. As with the baseline, this should be interpreted as a fixed-budget comparison rather than a fully converged training run.

| Iteration | Training Loss | Validation Loss | Validation Perplexity |
|-----------|---------------|-----------------|-----------------------|
| 0         | 10.8924       | 10.8927         | 53,784.35             |
| 150       | 7.0008        | 7.0628          | 1,167.66              |
| 300       | 6.4344        | 6.4896          | 658.28                |
| 450       | 6.1911        | 6.2530          | 519.56                |

|      |        |        |        |
|------|--------|--------|--------|
|      |        |        |        |
| 600  | 6.0146 | 6.1311 | 459.92 |
| 750  | 5.8600 | 5.9934 | 400.79 |
| 900  | 5.7695 | 5.9001 | 365.06 |
| 1050 | 5.7055 | 5.8337 | 341.61 |
| 1200 | 5.6478 | 5.7784 | 323.24 |
| 1350 | 5.6296 | 5.7808 | 324.03 |
| 1500 | 5.6153 | 5.7668 | 319.53 |

## 7. Comparison Against Baseline

| Metric                      | Baseline       | Weight Tying   | Change                 |
|-----------------------------|----------------|----------------|------------------------|
| Parameters                  | 32.16M         | 19.24M         | 40.16% fewer           |
| Final validation loss       | 5.7634         | 5.7693         | Almost same, +0.0059   |
| Final validation perplexity | 318.42         | 320.32         | Almost same, +1.90     |
| Checkpoint size             | 122.71 MB      | 73.44 MB       | 40.15% smaller         |
| Peak VRAM                   | 10.84 GB       | 10.69 GB       | Slightly lower         |
| Training time               | 156.42 sec     | 144.17 sec     | Faster by about 7.83%  |
| Training speed              | 158,641 tok/s  | 172,070 tok/s  | Faster by about 8.46%  |
| Generation latency          | 10.20 ms/token | 12.64 ms/token | Slower by about 23.97% |
| Energy                      | 10.78 Wh       | 9.87 Wh        | Lower by about 8.46%   |

## 8. Interpretation

Weight tying is a successful efficiency improvement for Logos. The experiment reduced the parameter count and checkpoint size by about 40% while preserving almost the same validation loss and perplexity as the baseline. This supports the hypothesis that the output projection matrix was largely redundant with the input token embedding matrix for this small decoder-only language model.

The result is especially important because the goal of Logos is not just to reduce loss, but to improve the capability-resource tradeoff. A model that achieves nearly the same validation loss with 40% fewer parameters is meaningfully more parameter-efficient and storage-efficient.

The only negative observation was generation latency. The weight-tied model produced lower generation speed in the final benchmark. This may be caused by runtime noise, memory layout differences, or benchmark instability. It should be retested with multiple generation benchmark runs before treating it as a firm conclusion.

## 9. Scientific Conclusion

Experiment 1 shows that weight tying is a strong candidate for the Logos efficiency stack. It produced a much smaller model and checkpoint while maintaining nearly the same validation quality. The result validates weight tying as a low-cost architectural improvement that should remain enabled in future Logos versions unless later experiments show a consistent disadvantage in generation latency or quality.

| Decision        | Recommendation                                                                 |
|-----------------|--------------------------------------------------------------------------------|
| Keep or reject? | Keep weight tying for future experiments                                       |
| Reason          | Large parameter and checkpoint reduction with nearly unchanged validation loss |
| Caution         | Recheck generation latency using repeated benchmark runs                       |
| Next experiment | GELU activation instead of ReLU                                                |

## 10. Next Experiment

The next clean experiment should change only the feed-forward activation function from ReLU to GELU while keeping all other settings identical. This will test whether a smoother activation commonly used in Transformer language models can improve validation loss without changing parameter count.

Experiment 2: GELU activation

Change only:

`nn.ReLU()` -> `nn.GELU()`

Keep fixed:

dataset, tokenizer, seed, batch size, context length, model depth, embedding size, heads, optimizer, scheduler, and max iterations

## One-Line Summary

Weight tying reduced Logos from 32.16M parameters to 19.24M parameters, making the model about 40% smaller while preserving nearly the same validation loss and perplexity under the same fixed training budget.